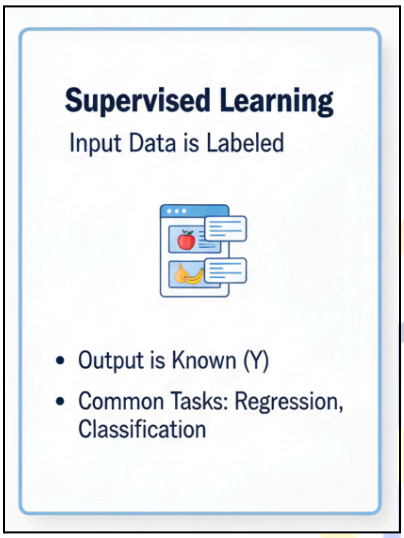
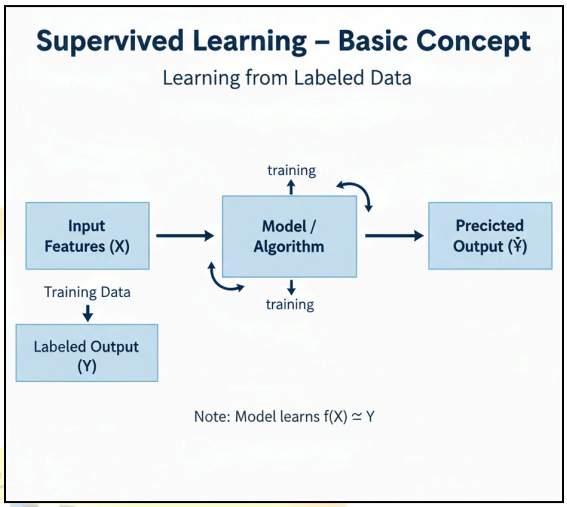
Supervised Learning

Introduction

Supervised learning is a branch of **Machine Learning (ML)** where the algorithm learns from **labeled data** — meaning that each training example includes both the input features and the correct output (target).

The system's goal is to learn a **mapping function** from inputs (X) to output (Y) so that it can predict the output for unseen data.





❖ Arrows showing training and prediction flow.

Formally,

Supervised learning finds a function $f(X) \approx Y$ using training data $(X_1, Y_1), (X_2, Y_2), \ldots, (X_n, Y_n)$.

Examples include:

- Predicting house prices based on size, location, etc.
- Classifying emails as spam or not spam.
- Predicting whether a customer will buy a product (Yes/No).

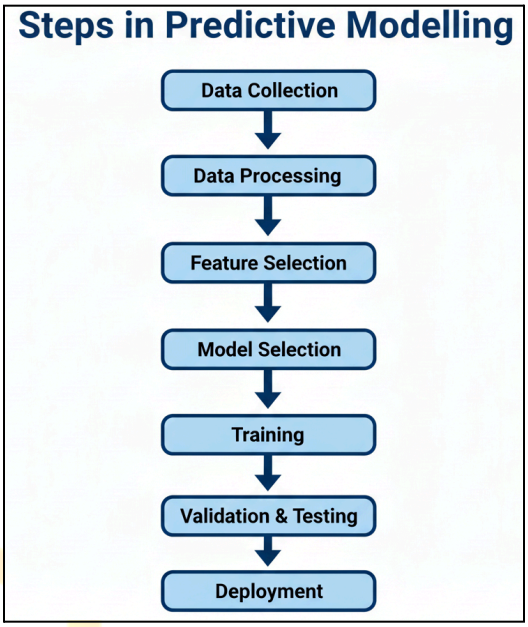
```
# Basic supervised learning workflow
from sklearn.linear_model import LinearRegression

# X = features, y = target labels
model = LinearRegression()
model.fit(X_train, y_train)  # Learning the mapping
predictions = model.predict(X_test)  # Applying to new data
```

---

1. Predictive Modelling

Predictive modeling refers to the process of building and using models to predict future or unknown outcomes based on known patterns from historical data.

1.1 Steps in Predictive Modeling



1. **Data Collection** – Gather labeled datasets (features + target).
2. **Data Preprocessing** – Handle missing data, normalize features, encode categorical variables.
3. **Feature Selection** – Choose the most relevant variables that influence the target.
4. **Model Selection** – Choose an appropriate supervised algorithm (e.g., Linear Regression, Decision Tree, SVM, etc.).
5. **Training** – Fit the model to the training data.
6. **Validation and Testing** – Evaluate the model on unseen data to test accuracy and generalization.
7. **Deployment** – Use the trained model for real-world predictions.

Example in Python (Simple Linear Regression):

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Sample Data
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([3, 4, 2, 5, 6])
# Model Training
model = LinearRegression()
model.fit(X, y)

# Prediction
new_value = np.array([[6]])
pred = model.predict(new_value)
print("Predicted value:", pred)
```

Explanation:

This example builds a simple linear model to predict a numeric outcome. The model learns from existing data and predicts the result for a new value (here, 6).

1.2 Types of Supervised Learning

- **Regression:**
  Predicts continuous numerical values.
  Example: Predicting temperature or sales revenue.
  Algorithms: *Linear Regression, Decision Tree Regressor, Random Forest Regressor.*

- **Classification:**
  Predicts categorical outcomes or class labels.
  Example: Predicting whether an email is “spam” or “not spam”.

Algorithms: *Logistic Regression, K-Nearest Neighbors, Support Vector Machines, Naive Bayes.*

```
# Complete workflow example
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Train model
model = LogisticRegression()
model.fit(X_train, y_train)
# Evaluate
y_pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
```

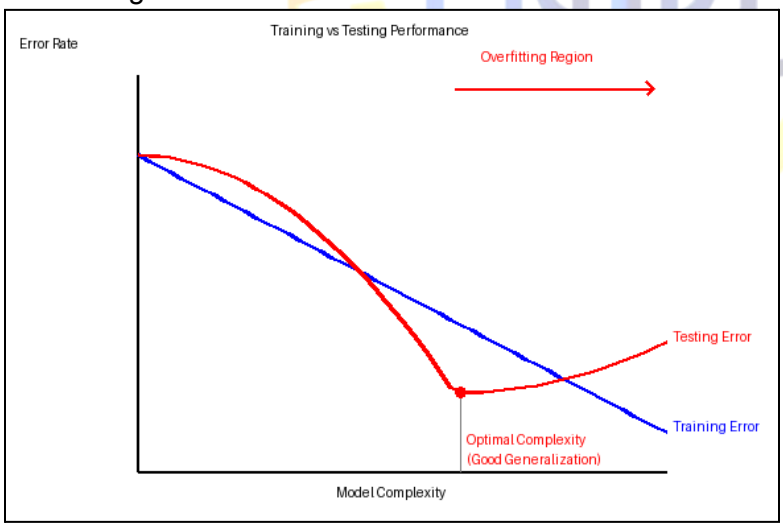
Hence, predictive modeling provides a systematic framework to forecast continuous (regression) or categorical (classification) outcomes.

---

2. Data Generalizability

Generalizability refers to a model's ability to perform well on new, unseen data — not just the data it was trained on.

A model with good generalization captures **true patterns** from the data rather than memorizing the training set.



"Lowest point of test error = good generalization."

2.1 Why Generalization Matters

- Real-world data is often slightly different from training data.
- If a model memorizes the training examples, it may fail on new data (this problem is called *overfitting*).
- Therefore, the goal of supervised learning is not to get perfect accuracy on training data, but to perform **consistently well** on test data.

2.2 Improving Generalization

- Use **cross-validation**.
- Ensure **diverse and representative data**.
- Apply **regularization** (penalize overly complex models).
- Use **early stopping** during training.
- Avoid excessive number of features.

---

3. Cross-Validation

3.1 Definition

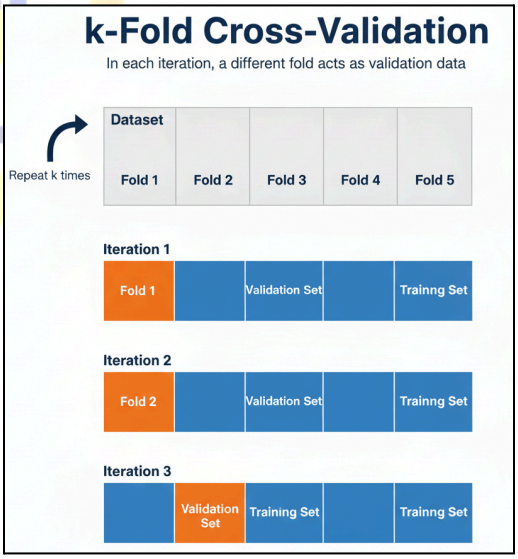
Cross-validation is a statistical method used to evaluate how well a model will generalize to an independent dataset.

It splits the available data into multiple parts to ensure that every observation is used for both training and validation.

3.2 Common Methods

- **Hold-Out Validation:**
  The dataset is divided into two parts: training and testing (for example, 80% train, 20% test).
  Simple but may depend on how the data was split.

- **k-Fold Cross-Validation:**
  The dataset is divided into *k* equal parts (folds).
  The model is trained on *k-1* folds and tested on the remaining fold.
  This process is repeated *k* times, and the average performance is taken.
  Example: k = 5 or k = 10.




```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import LinearRegression
import numpy as np

X, y = data.drop('target', axis=1), data['target']
model = LinearRegression()
scores = cross_val_score(model, X, y, cv=5)
print("Mean Accuracy:", np.mean(scores))
```

- **Leave-One-Out Cross-Validation (LOOCV):**
  Each observation is used once as a test set, while the remaining are used for training.
  Very accurate but computationally expensive.
- **Stratified Cross-Validation:**
  Ensures that each fold has the same proportion of class labels (useful for imbalanced datasets).

3.3 Purpose
Cross-validation helps to:
- Detect **overfitting** or **underfitting**.
- Select the best model or hyperparameters.
- Get a reliable estimate of model accuracy on unseen data.

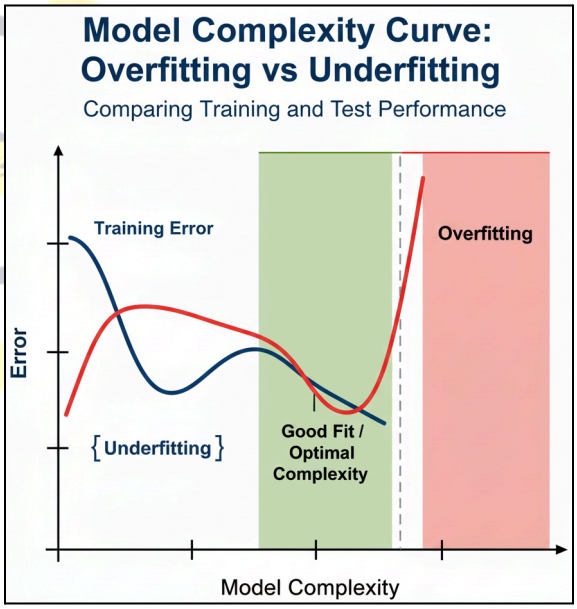
Advantages:
- Provides a better estimate of model performance.
- Reduces the effect of random data splits.
- Helps in tuning hyperparameters more effectively.

---

4. Overfitting and Underfitting

5.1 Overfitting
Overfitting occurs when a model learns **too much detail** or **noise** from the training data, performing extremely well on it but poorly on new data.

- **Symptoms:**
  - High training accuracy
  - Low test accuracy
  - Model too complex (too many features or parameters)
- **Causes:**
  - Insufficient or unrepresentative data
  - Too many features relative to sample size
  - Complex models (deep trees, large neural networks)
- **Prevention:**
  - Use simpler models
  - Apply **regularization** (L1, L2)
  - Use **cross-validation**
  - Collect more training data
  - Use **dropout** (in neural networks)
  - Perform **early stopping**



Example:
Imagine fitting a very high-degree polynomial through a few scattered points.
The curve will pass exactly through each point but will oscillate wildly — failing to predict new data correctly.

Techniques to Reduce Overfitting:
- **Cross Validation:** Ensures model performs consistently on unseen folds.
- **Regularization:** Adds penalty terms to limit model complexity (L1, L2 regularization).
- **Early Stopping:** Stops training when validation performance stops improving.
- **Simpler Models:** Use fewer parameters or simpler algorithms.
- **More Data:** Increasing dataset size improves generalization.
- **Feature Selection:** Remove irrelevant or redundant features.

Example (Regularization in Linear Model):

```
from sklearn.linear_model import Ridge
import numpy as np

# Sample Data
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])

# Ridge Regression (L2 Regularization)
model = Ridge(alpha=0.5)
model.fit(X, y)
print("Predictions:", model.predict(X))
```

Explanation:
Ridge regression prevents overfitting by penalizing large coefficient values, thus keeping the model simpler and more generalizable.

5.2 Underfitting

Underfitting occurs when a model is too simple and fails to capture patterns in the data.
It performs poorly both on training and test sets.

- **Causes:**
  - Too few features
  - Model too simple (e.g., linear model for nonlinear data)
  - Inadequate training time
- **Solution:**
  - Use a more complex model
  - Add relevant features
  - Train longer (for neural networks)

While overfitting learns noise, underfitting fails to learn the signal.

Overview:

Concept	Definition	Purpose
Supervised Learning	Model learns mapping from inputs to outputs using labeled data.	Predict unseen outcomes.
Predictive Modelling	Process of building and validating models for prediction.	Forecast or classify outcomes.
Data Generalizability	Model's ability to perform well on new, unseen data.	Ensures model is not overfitted.
Cross-Validation	Method to assess how results will generalize to an independent dataset.	Prevents overfitting, selects best model.
Overfitting	Model fits training data too closely and performs poorly on new data.	Avoid by regularization and validation.